

Anhang B

Die C-Standardbibliothek

Die Header-Dateien der *C-Standardbibliothek* sind hier vollständig aufgezählt, nicht jedoch die der enthaltenen Elemente. Hier sollen nur die (subjektiv) wichtigsten Konstanten, Typen, Funktionen und Variablen aufgelistet werden. Auf UNIX/LINUX-Systemen sollte `man 3 fkt` bzw. `man 3 datei` die Dokumentation zur Funktion *fkt* bzw. zur Headerdatei *datei.h* der Standardbibliothek liefern.

- `assert.h`

debugging-Hilfen:

Makro `NDEBUG` - Funktion `assert`

- `ctype.h`

Funktionen zur Klassifizierung/Umwandlung von Zeichen

Argument und Funktionswert der Funktionen sind jeweils vom Typ `int`. Hat das übergebene Argument den Wert EOF, so ist der Funktionswert bei allen Funktionen EOF. Wird die Funktion mit einer `char`-Variablen als Argument aufgerufen, so wird deren Wert implizit in einen `int`-Wert umgewandelt und es werden folgende Werte zurückgeliefert:

- `int isalnum(int c)`
 - ungleich 0, falls `c` ein alphanumerisches Zeichen repräsentiert (Ziffer oder Buchstabe), 0 sonst.
- `int isalpha(int c)`
 - ungleich 0, falls `c` Buchstaben repräsentiert, 0 sonst.
- `int iscntrl(int c)`
 - ungleich 0, falls `c` ein Steuerzeichen repräsentiert (siehe Zeichensatz), 0 sonst
- `int isdigit(int c)`
 - ungleich 0, falls `c` Ziffer repräsentiert, 0 sonst.
- `int isgraph(int c)`
 - ungleich 0, falls `c` druckbares Zeichen (ohne Leerzeichen) repräsentiert, 0 sonst.
- `int islower(int c)`
 - ungleich 0, falls `c` Kleinbuchstaben repräsentiert, 0 sonst.
- `int isprint(int c)`
 - ungleich 0, falls `c` druckbares Zeichen (mit Leerzeichen) repräsentiert, 0 sonst.
- `int ispunct(int c)`
 - ungleich 0, falls `c` ein druckbares Sonderzeichen repräsentiert, 0 sonst.

- `int isspace(int c)`
 - ungleich 0, falls `c` *Leerraum* repräsentiert (also Leerzeichen oder eines der Zeichen `\f`, `\n`, `\r`, `\t`, `\v`), 0 sonst
- `int isupper(int c)`
 - ungleich 0, falls `c` Großbuchstaben repräsentiert, 0 sonst.
- `int isxdigit(int c)`
 - ungleich 0, falls `c` hexadezimale Ziffer repräsentiert, 0 sonst.
- `int tolower(int c)`
 - Falls der Argumentwert ein Wert aus dem Bereich `'A'` bis `'Z'` ist, so wird der entsprechende „Kleinbuchstabenwert“ (aus `'a'` bis `'z'`) zurückgegeben. Anderenfalls wird der Argumentwert zurückgegeben.
- `int toupper(int c)`
 - wie `tolower` nur umgekehrt: zu „Kleinbuchstabenwert“ wird (soweit möglich) entsprechender „Großbuchstabenwert“ zurückgegeben
- **errno.h**
Fehlerbehandlung für mathematische Funktionen:
Makro `EDOM` - Makro `ERANGE` - globale Variable `errno`
- **float.h**
Angaben zu den Gleitkommazahlwertebereichen:
`FLT_MAX` -
`FLT_MIN` -
`DBL_MAX` -
`DBL_MIN` -
`LDBL_MAX` -
`LDBL_MIN`
- **iso646.h**
alternative Schreibweisen für logische Operatoren
- **limits.h**
Angaben zu den Ganzzahlwertebereichen:
`CHAR_BIT` - `CHAR_MAX` - `CHAR_MIN` - `SCHAR_MAX` - `SCHAR_MIN` - `UCHAR_MAX` - `SHRT_MAX` - `SHRT_MIN` - `USHRT_MAX` - `INT_MAX` - `INT_MIN` - `UINT_MAX` - `LONG_MAX` - `LONG_MIN` - `ULONG_MAX`
- **locale.h**
Sprach-, Schrift-, Zeitrechnungs- und währungsspezifische Angaben
- **math.h**
Gleitkomma-Arithmetik:
Makro `HUGE_VAL`, Funktionen: `fabs` - `ceil` - `floor` -
`acos` - `asin` - `atan` -
`cos` - `sin` - `tan` - `cosh` - `sinh` - `tanh` - `exp` -
`log` - `log10` -
`pow` - `sqrt`
- **setjmp.h**
Sprünge in andere Programmteile

-
- **signal.h**
Signal(Interrupt)behandlung
 - **stdarg.h**
Variable Argumentlisten für Funktionen mit einer variablen Anzahl von Parametern (wie z.B. `printf` und `scanf`):
`va_list` - `va_start` - `va_arg` - `va_end`
 - **stddef.h**
Definitionen von einfachen Typen und Werten
 - `NULL`
- der „Nullzeiger“, eine benannte Konstante vom Zeigertyp deren Wert verschieden ist von jedem echten Zeigerwert
 - `size_t`
- vorzeichenloser ganzzahliger Typ dessen Wertebereich alle zulässigen Speicherbereichsgrößen umfasst (m.a.W.: Menge der Werte, die vom `sizeof`-Operator (in der jeweiligen Implementation) zurückgeliefert werden können)
 - **stdio.h**
Ein- und Ausgabe-Funktionen (siehe die Kapitel des Skripts zur Ein- und Ausgabe)
 - **stdlib.h**
Umwandlung von Strings zu Zahlwerten:
`atof` - `atoi` - `atol` - `strtod` - `strtol` - `strtoul`
Pseudozufallsgenerator:
`RAND_MAX` - `rand` - `srand`
dynamische Speicherverwaltung:
`malloc` - `calloc` - `realloc` - `free`
ganzzahlige Mathematik:
`div_t` - `ldiv_t` - `abs` - `div` - `labs` - `ldiv`
Programmabbruch:
`EXIT_FAILURE` - `EXIT_SUCCESS` - `exit` - `abort` - `atexit`
Systemaufrufe:
`system` - `getenv`
Algorithmen:
`bsearch` - `qsort`
Umwandlungen zwischen verschiedenen Zeichensätzen
 - **string.h**
Stringbearbeitung
 - `char *strerror(int Kennzahl);`
- liefert den Zeiger auf den Anfang des Strings, der die Klarschrift-Fehlermeldung zum Fehler mit dem Fehlercode `Kennzahl` enthält
 - `size_t strlen(const char *s);`
- liefert die Anzahl der Zeichen des Strings, auf dessen Anfang `s` zeigt. Das Stringende-Zeichen des Strings wird dabei nicht mitgezählt.

- `char *strcpy(char *s1, const char *s2);`
 - kopiert (byteweise) die Zeichen des Strings auf dessen Anfang `s2` zeigt (inklusive Stringende-Zeichen) in einen Speicherbereich auf dessen Anfang `s1` zeigt. Der Wert von `s1` wird als Funktionswert zurückgeliefert.
- `char *strncpy(char *s1, const char *s2, size_t n);`
 - wie `strcpy`, jedoch werden *genau* `n` Zeichen geschrieben. Außerdem wird das Kopieren beendet, sobald das Stringende-Zeichen im Quellstring gefunden wurde und ergänzt danach nur noch (binäre) Nullen. Das bedeutet insbesondere: Wenn der zu kopierende String aus `n` oder mehr Zeichen besteht, wird der kopierte String nicht durch ein Stringende-Zeichen abgeschlossen. Der Wert von `s1` wird als Funktionswert zurückgeliefert.
- `char *strcat(char *s1, const char *s2);`
 - kopiert den String auf dessen Anfang `s2` zeigt hinter den String auf dessen Anfang `s1` zeigt. Das Stringende-Zeichen von `s1` wird dabei überschrieben. Hinter dem letzten kopierten Zeichen wird das Stringende-Zeichen automatisch angehängt. Der Wert von `s1` wird als Funktionswert zurückgeliefert.
- `char *strncat(char *s1, const char *s2, size_t n);` - wie `strcat`, jedoch wird das Kopieren vorzeitig beendet, sobald `n` Zeichen übertragen wurden.
- `int strcmp(const char *s1, const char *s2)`
 - liefert einen `int`-Wert kleiner, gleich oder größer Null, je nachdem, ob der String, auf dessen Anfang `s1` zeigt, *lexikographisch* kleiner, gleich oder größer ist als der String, auf dessen Anfang `s2` zeigt. [Ein String heißt lexikographisch kleiner als ein anderer, wenn in ihm an der ersten Unterscheidungsstelle ein kleinerer `char`-Wert steht als im anderen String.
- `int strncmp(const char *s1, const char *s2, size_t n);`
 - wie `strcmp`, jedoch werden höchstens `n` Zeichen verglichen
- `char *strchr(const char *s, int c)`
 - liefert Zeiger auf das zuerst gefundene Vorkommen des Zeichens `c` in dem String, auf dessen Anfang `s` zeigt bzw. den Nullzeiger, falls das Zeichen nicht gefunden wurde.
- `strrchr`
 - wie `strchr`, jedoch wird beginnend beim Stringende-Zeichen in Richtung Anfang gesucht.
- `strspn`
- `strcspn`
- `strpbrk`
- `char *strstr(const char *s1, const char *s2)`
 - sucht das erste Vorkommen des Strings auf dessen Anfang `s2` zeigt, im String, auf dessen Anfang `s1` weist (dabei wird das Stringende-Zeichen nicht mit verglichen). Der Funktionswert ist der Zeiger auf den Anfang der gefundenen Teilfolge bzw. der Nullzeiger.
- `strtok`

Speicherbearbeitung

- `memchr`
- `memcmp`

- `void *memcpy(void *s1, const void *s2, size_t n)`
 - kopiert die ersten `n` Bytes eines Speicherbereichs auf dessen Anfang `s2` zeigt in die ersten `n` Bytes des Speicherbereichs auf dessen Anfang `s1` zeigt. Funktionswert ist `s1`.
 - `memmove`
 - `memset`
- `time.h`
 - Zeitmessung:
`CLOCKS_PER_SEC` - `clock_t` - `clock`
 - Datums- und Uhrzeitbestimmung:
`time_t` - `time` - `difftime` - `struct tm` - `gmtime` - `localtime` - `asctime` - `ctime`
- `wchar.h`
 - Umwandlung von Strings zu Zahlwerten für den erweiterten Zeichensatz
 - String- und Speicherbearbeitung für den erweiterten Zeichensatz
 - Ein- und Ausgabe für den erweiterten Zeichensatz
- `wctype.h`
 - Zeichenuntersuchung für den erweiterten Zeichensatz

Abbildungsverzeichnis

7.1	Türme von Hanoi	77
11.1	Speicheranordnung eines Vektors mit 6 Komponenten	112
11.2	Speicheranordnung einer (2×3) -Matrix	112
14.1	Verkettete Liste	156
A.1	Beispiel eines UNIX-Filesystems	179

Tabellenverzeichnis

2.1	Vorgeschriebene Wertebereiche für Ganzzahltypen	11
2.2	Escapesequenzen	13
2.3	Schlüsselwörter von C	18
3.1	Kennbuchstaben für Ausgabeformate	23
3.2	Kennbuchstaben für Eingabeformate	25
4.1	Wahrheitstabeln für logische Operatoren	37
4.2	Prioritäten der Operatoren	40
4.3	Hierarchie der impliziten Typumwandlung	43
5.1	Funktionen zur Klassifizierung von Zeichen	52
9.1	Mathematische Funktionen mit einem Argument	100
16.1	Optionen zum Öffnen von Dateien mit fopen	166

Quelltextbeispiele

1.1	Unser Erstes Beispiel	5
3.1	einfache formatierte Eingabe/Ausgabe	21
3.2	Wiederholt Zahlen einlesen und ausgeben mit einer Schleife	26
3.3	Zahlen kopieren mit Vorschüben	27
3.4	Eingabe bis zum Ende der Eingabe (EOF) verarbeiten	28
3.5	Auswahl aus Alternativen mit <code>if</code> und <code>else</code>	29
3.6	Arbeit mit Vektoren: Berechnen eines Skalarproduktes	31
4.1	Inkrement-Operator	35
5.1	Kopieren von 10 Zahlen (Eine Zahl pro Zeile)	49
6.1	Berechnung von Kreisumfang oder Fläche	57
6.2	Berechnung von Kreisumfang oder Fläche mit <code>switch</code>	60
6.3	Auswahl aus Alternativen (neu)	62
6.4	Arbeit mit Vektoren: Skalarprodukt mit <code>for</code> -Schleife	63
6.5	Auswahl aus Alternativen (mit <code>break</code>)	65
6.6	Ziffernübereinstimmungen	67
7.1	Invertieren eines Strings	73
7.2	Türme von Hanoi rekursiv	78
8.1	Invertieren eines Strings mit Funktionen	83
8.2	Stapelverwaltung	86
8.3	Stapelverwaltung (modularisiert I): <code>stapel.h</code>	88
8.4	Stapelverwaltung (modularisiert I): <code>stapel.c</code>	88
8.5	Stapelverwaltung (modularisiert I): <code>stapeltest.c</code>	89
8.6	<code>makefile</code> für Stapelverwaltung	91
8.7	Stapelverwaltung (modularisiert II): <code>stapel.c</code>	92
8.8	Stapelverwaltung (modularisiert III): <code>stapel.h</code>	93
8.9	Stapelverwaltung (modularisiert III): <code>stapel.c</code>	93
12.1	Verwendung von Zeigern auf Funktionen und <code>typedef</code>	136
13.1	Auflisten der letzten Zahlen der Eingabe	143
13.2	Matrizen mit beliebigen Dimensionen bereitstellen	147
14.1	Arbeiten mit Strukturen	151
14.2	Arbeiten mit verschachtelten Strukturen und Zeigern auf Strukturen	153
16.1	Dateiverarbeitung: Konkatenation	166

Literaturverzeichnis

- [1] C Programming Language. Wikipedia-Eintrag: http://en.wikipedia.org/w/index.php?title=C_programming_language&oldid=40429557.
- [2] GNU Compiler Collection (GCC). Internet-Website: <http://gcc.gnu.org/>.
- [3] Joachim Goll, Ulrich Bröckl, and Manfred Dausmann. *C als erste Programmiersprache*. Teubner, 2003.
- [4] Samuel P. Harbison and Guy L. Steele Jr. *C. A Reference Manual*. Prentice Hall, 2002.
- [5] Sammlung von Standardisierungsdokumenten. Internet-Website: <http://www.open-std.org/JTC1/SC22/WG14/www/standards>, Juni 2005.
- [6] ISO/IEC 9899:TC2. Internet-Dokument: <http://www.open-std.org/jtc1/sc22/wg14/www/docs/n1124.pdf>, Mai 2005. ISO/IEC.
- [7] Brian W. Kernighan and Dennis M. Ritchie. *The C Programming Language*. Prentice Hall, 2nd edition, 1988.
- [8] Linksammlung zur C-Programmierung. Internet-Website: <http://www.lysator.liu.se/c/>, 2001.
- [9] Martin Lowes and Augustin Paulik. *Programmieren mit C. ANSI Standard*. Teubner, 1999.

Stichwortverzeichnis

Symbole

#define	16, 106
#elif	107
#else	107
#if	107
#ifdef	109
#ifndef	109
#include	106
#undef	109

A

abort	102
Adressoperator	119
Aktualargumente	72
ANSI-Standard	1
Argumente	72
arithmetische Operatoren	34
array	<i>siehe</i> Felder
ASCII-Code	12
Aufzählungskonstanten	15
Ausdrucksanweisungen	55
Ausdrücke	33
ausführbares Programm	3
Auslöschung	163
auto	95

B

bedingte Compilation	105
bedingter Ausdruck	38
benannte Konstanten	15
Bezeichner	17
Bibliotheksdateien	101
Binärzahlen	9
Block	55
boolean	36
break	59, 65
Byte	10

C

call by reference	73, 123
-------------------------	---------

call by value	72, 123
case	58
cast	<i>siehe</i> Typumwandlung
Castoperatoren	44
char	12
character	12
Compiler	2, 183
const	75, 122
continue	66

D

Datei	178
Dateivariablen	165
Dateiverarbeitung	165
default	58
Dekrementierung	35
Dereferenzierung	119
Dereferenzierungsoperator	119
Dezimalzahlen	9
Direktiven	105
Division	34
Divisionsrest	34
do	60
double	14
dynamischer Speicher	116, 141

E

echo	27
Editor	183
EDOM	101
else	56
End Of File	28
enum	15
ERANGE	101
errno	101
Escapesequenzen	13
exit	102
extern	85
extern	93

- F**
- Felder 30, 111
 - Elemente 111
 - Initialisierung 116
 - Komponenten 111
 - float 14
 - for 62
 - Formatbeschreiber 22
 - Ausgabe 168
 - Eingabe 171
 - formatierte Ein-/Ausgabe 21
 - free 142
 - Funktionen 69
 - Argument 71
 - Aufruf 71
 - Definition 69
 - Deklaration 69
 - Prototyp 69
 - Funktionsheader 70
 - Funktionswert 71
 - Funktionszeiger 136
- G**
- gepufferte Eingabe 27
 - geschweifte Klammern 55
 - getenv 102
 - Gleitkommazahlen 14
 - global 92
 - goto 64
 - graphische Zeichen 3
- H**
- Hauptmodul 89
 - Hauptprogramm 4
 - Headerdatei 88
 - Heap 141
 - Hexadezimalzahlen 9
- I**
- if 55
 - Index 30, 111
 - Indexüberschreitung 113
 - Initialisierung 17
 - Felder 116
 - Strukturen 151
 - Inkrementierung 35
 - int 10
 - integer 10
- intern** 85
- Iteration** 76
- K**
- Kommaoperator 39
 - Kommentar 5
 - Konkatenation 133
- L**
- last in first out (lifo) 85
 - lazy evaluation 37
 - leere Anweisung 55
 - Linker 3, 183
 - logische Operatoren 37
 - lokal 92
 - long 10
 - long double 14
 - loop *siehe* Schleifen
- M**
- main 4
 - Makros 106
 - Expandierung 106
 - malloc 142
 - Matrizen 30, 111
 - Mindestschränken 10
 - Minus 34
 - Module 88
 - Modulo-Operator 34
 - Multiplikation 34
- N**
- Namen 17
 - Nebeneffekt 41
 - normalisierte Darstellung 160
 - Null-Zeichen 47
 - Nullzeiger 127
- O**
- Objekt-Programm 3
 - Objektdatei 183
 - Oktalzahlen 9
 - Operanden 33
 - Operatoren 33
 - * 34
 - * (Dereferenzierungsoperator) 119
 - + (Inkrementierung) 35
 - , (Kommaoperator) 39

- 34
- (Dekrementierung) 35
- > 153
- .. (Punktoperator) 151
- / 34
- < 36
- <= 36
- = 33
- == 36
- > 36
- >= 36
- ? : (bedingter Ausdruck) 38
- % (Modulo-Operator) 34
- & (Adressoperator) 119
- && (logisches UND) 37
- Ordnungszahlen 12
- overflow *siehe* Überlauf
- P**
- Parameter 72
- Pfad 178
- Plus 34
- pointer *siehe* Zeiger
- portabel 2
- Präprozessor 5, 105
- Präprozessor-Direktiven 6
- printf 22
- Prioritäten 39
- Punktoperator 151
- R**
- rand 103
- register 96
- Rekursion 4
 - direkte 76
 - indirekte 76
- return 71
- runde Klammer 33
- Rundungsfehler 162
- S**
- scanf 24
- Schlüsselwörter 18
- short 10
- signed 11
- sizeof 45
- Speicher
 - dynamisch 141
 - statisch 141
- Speicherabbildungsfunktion 113, 143
- Speicherblöcke 139
- Sprünge 64
 - break 65
 - continue 66
- srand 103
- sscanf 143
- Stack 141
- Standard-Headerdateien 99
- Standardausgabegerät 21, 183
- Standardbibliothek 1, 99, 185
- Standardeingabegerät 21, 183
- Standardtypen 10
- static 92, 95
- statischer Speicher 141
- String 14
- struct 149
- Strukturen 149
 - Initialisierung 151
- switch 58
- Syntaxeinfärbung 2
- system 102
- T**
- typedef 19, 136
- Typumwandlung 42
 - implizit 42
- U**
- Überlauf 159
- underflow *siehe* Unterlauf
- union 157
- UNIX 177
 - home directory 180
 - Jokerzeichen 182
 - login 177
 - Metazeichen 182
 - root 177
 - wild cards 182
 - working directory 179
- UNIX-Befehle
 - cd (change directory) 180
 - chmod (change mode) 181
 - cp (copy) 181
 - lpq (lineprinter queue) 182
 - lprm (lineprinter remove) 183
 - lpr (lineprinter) 182
 - ls (list) 180
 - man (manual) 178

<code>mkdir</code> (make directory)	181
<code>mv</code> (move)	181
<code>pwd</code> (print working directory)	179
<code>rmdir</code> (remove directory)	181
<code>rm</code> (remove)	181
<code>unsigned</code>	11
Unterlauf	159

V

Variablen	16
automatisch	94
statisch	94
Vektoren	30, 111
Verbunde	157
Vergleichsoperatoren	36
verkettete Listen	156
Verschattung	85
Verzeichnisse	178
Verzweigungen	64
<code>void</code>	69
<code>volatile</code>	97

W

Wahrheitstafeln	37
Wertzuweisung	33
<code>while</code>	60
white spaces	3
Wortstruktur	10

Z

Zeichenkettenkonstante	14
Zeiger	120
Zeiger auf Zeiger	132
Zeigerarithmetik	124
Zeigerausdruck	120
Zeigerparameter	120
Zeigervariablen	121
Zufallszahlen-Generator	103
Zuweisungsoperator	33
zusammengesetzter	38